

Application of Genetic Algorithms for PID-LQR Tuning in Quarter-Car Suspension Control

Izma Alhazmi Herdian
Teknik Fisika
Institut Teknologi Bandung
Bandung, Indonesia
Email: 13321027@mahasiswa.itb.ac.id

I. PENDAHULUAN

Di awal abad ke-21, perkembangan pesat kecerdasan buatan (AI) dan metode numerik telah mengubah pendekatan penyelesaian masalah di berbagai bidang teknik, termasuk mekatronika. Insinyur mekatronika kini dituntut untuk menguasai keterampilan dalam analisis sistem, desain, kendali, dan pemodelan sistem sebagai bagian dari kompetensi inti mereka. Pemodelan sistem, khususnya, menjadi fondasi penting dalam memahami dinamika sistem serta dalam merancang dan mengendalikan sistem yang kompleks [1].

Seiring dengan kemajuan teknologi dalam era Industri 4.0, kebutuhan akan algoritma pemrosesan sinyal yang efisien terus meningkat. Algoritma berbasis pemodelan stokastik, seperti filter Schur adaptif, telah menunjukkan keunggulan dalam menyesuaikan perubahan sinyal secara dinamis, memberikan solusi yang lebih akurat dalam berbagai kondisi tidak pasti. Di bidang otomotif, penelitian mulai berfokus pada pengembangan sistem suspensi aktif, yang dirancang untuk meningkatkan kenyamanan dan stabilitas kendaraan. Suspensi aktif memungkinkan penyesuaian gaya secara real-time, merespons kondisi jalan yang bervariasi, sehingga memenuhi tuntutan industri otomotif yang semakin menekankan pada kenyamanan dan keselamatan [1].

Dalam pengembangan sistem kontrol untuk suspensi aktif, *Linear Quadratic Regulator* (LQR) menjadi salah satu metode yang populer karena kemampuannya dalam mengoptimalkan kinerja sistem. Namun, desain parameter LQR yang optimal sering kali menjadi tantangan, terutama untuk sistem yang kompleks seperti model sistem seperempat mobil (*quarter car model*) dengan dua derajat kebebasan. Untuk mengatasi tantangan ini, Genetic Algorithm muncul sebagai solusi potensial yang mampu melakukan pencarian parameter secara global.

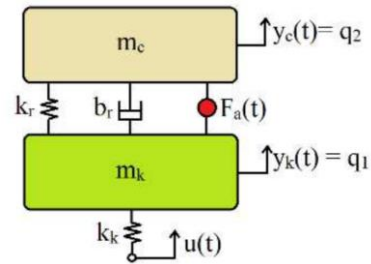
Genetic Algorithm adalah salah satu teknik optimasi berbasis evolusi yang meniru proses seleksi alam. Dengan memanfaatkan prinsip-prinsip seperti mutasi, seleksi, dan crossover, algoritma ini dapat mencari parameter optimal untuk LQR dalam ruang pencarian yang luas. Penggunaan Genetic Algorithm dalam tuning LQR untuk sistem suspensi aktif memungkinkan peningkatan kinerja kendaraan pada berbagai kondisi jalan, menjadikannya solusi yang menjanjikan di bidang otomotif.

Penelitian ini bertujuan untuk mengimplementasikan Genetic Algorithm dalam tuning kontrol LQR untuk model seperempat mobil. Pendekatan ini diharapkan dapat meningkatkan performa suspensi aktif dalam meredam getaran kendaraan, sehingga memberikan kontribusi signifikan terhadap pengembangan teknologi kendaraan masa depan.

II. PEMODELAN SISTEM [1]

A. Persamaan Gerak Suspensi Kendaraan dengan Aktuator

Untuk melakukan pemodelan dalam ruang keadaan, diperlukan persamaan gerak dalam bentuk matriks dari model suspensi. Model ini dirancang secara detail dengan mempertimbangkan elastisitas ban dan berat roda, seperti yang ditunjukkan pada **Error! Reference source not found..**



Gambar 1. Model Suspensi Kendaraan

Merujuk pada Gambar 1 persamaan gerak dapat dirumuskan dengan menggunakan persamaan Lagrange jenis kedua atau hukum kedua Newton sebagai berikut.

$$\begin{aligned} I. m_k \cdot \ddot{y}_k &= \Sigma F_k \\ II. m_c \cdot \ddot{y}_c &= \Sigma F_c \end{aligned} \quad (1)$$

$$\begin{aligned} I. m_k \cdot \ddot{y}_k &= k_r \cdot (u - y_k) - k_r \cdot (y_k - y_c) - b_r \cdot (\dot{y}_k - \dot{y}_c) - F_a \\ II. m_c \cdot \ddot{y}_c &= k_r \cdot (y_k - y_c) + b_r \cdot (\dot{y}_k - \dot{y}_c) + F_a \end{aligned} \quad (2)$$

$$\begin{aligned} I. \ddot{y}_k &= \frac{k_r \cdot (u - y_k) - k_r \cdot (y_k - y_c) - b_r \cdot (\dot{y}_k - \dot{y}_c) - F_a}{m_k} \\ II. \ddot{y}_c &= \frac{k_r \cdot (y_k - y_c) + b_r \cdot (\dot{y}_k - \dot{y}_c) + F_a}{m_c} \end{aligned} \quad (3)$$

$$\begin{aligned} I. \ddot{y}_k &= \frac{k_r \cdot u - k_r \cdot y_k}{m_k} - \frac{k_r \cdot y_k - k_r \cdot y_c}{m_k} - \frac{b_r \cdot \dot{y}_k - b_r \cdot \dot{y}_c}{m_k} - \frac{F_a}{m_k} \\ II. \ddot{y}_c &= \frac{k_r \cdot y_k - k_r \cdot y_c}{m_c} + \frac{b_r \cdot \dot{y}_k - b_r \cdot \dot{y}_c}{m_c} + \frac{F_a}{m_c} \end{aligned} \quad (4)$$

Pemilihan variabel keadaan diperlukan untuk merumuskan model state space berikut.

$$x_1 = y_k \quad \dot{x}_1 = \dot{y}_k = x_2 \quad (5)$$

$$x_2 = \dot{y}_k \quad \dot{x}_2 = \ddot{y}_k \quad (6)$$

$$x_3 = y_c \quad \dot{x}_3 = \dot{y}_c = x_4 \quad (7)$$

$$x_4 = \dot{y}_c \quad \dot{x}_4 = \ddot{y}_c \quad (8)$$

B. Representasi State Space Suspensi Kendaraan dengan Aktuator

Struktur umum model ruang keadaan untuk persamaan input dan output adalah sebagai berikut.

$$\dot{x} = A.x + B.u \quad (9)$$

$$y = C.x \quad (10)$$

Dengan menggunakan struktur pada persamaan (9) dan (10) dan menggantikan variabel yang sesuai ke dalam matriks model, diperoleh persamaan berikut.

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \\ \dot{x}_3 \\ \dot{x}_4 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ -\frac{k_k - k_r}{m_k} & -\frac{b_r}{m_k} & \frac{k_r}{m_k} & \frac{b_r}{m_k} \\ 0 & 0 & 0 & 1 \\ k_r & m_c & -\frac{k_r}{m_c} & -\frac{b_r}{m_c} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} \quad (11)$$

$$+ \begin{bmatrix} 0 & 0 \\ -\frac{1}{m_k} & \frac{k_k}{m_k} \\ 0 & 0 \\ \frac{1}{m_c} & 0 \end{bmatrix} \begin{bmatrix} F_a \\ u \end{bmatrix}$$

$$y = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} \quad (12)$$

Matriks output C ditetapkan sebagai $\begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$ untuk memfokuskan pada keluaran y_c dan percepatannya $\ddot{y}_c$.

C. Parameter Suspensi Kendaraan dengan Aktuator

Untuk penelitian ini, parameter model suspensi seperempat mobil yang digunakan adalah milik BMW 530i, dengan nilai performanya tercantum pada Tabel 1.

Tabel 1. BMW 530i Suspension Parameters

Simbol	Variabel	Nilai
k_r	Suspension Spring Constant [kN/m]	30
b_r	Suspension Damping Constant [ks/m]	1450
k_k	Tire Spring Constant [kN/m]	340
m_c	Mass of Vehicle Body [kg]	408
m_{us}	Mass of Wheel [kg]	48.3

III. DESAIN SISTEM KONTROL [2]

Proportional Integral Derivative - Linear Quadratic Regulator (PID – LQR) adalah kombinasi dari pengendalian kontrol PID dan LQR untuk mendesain pengontrol yang optimal dan stabil. Dalam pendekatan ini, parameter K_p , K_i , dan K_d dari kontrol PID ditentukan menggunakan teknik LQR. PID mengendalikan kesalahan keluaran aktual dengan referensi baik secara langsung, menjumlahkan kesalahan dari waktu ke waktu, dan memperkirakan perubahan kesalahan. LQR merupakan metode untuk mendesain kontrol optimal pada sistem dinamis linear, dengan tujuan meminimalkan fungsi biaya yang menggabungkan kesalahan state dan energi kontrol.

A. PID

1) Kontrol PID

Fungsi kontrol PID ini menggabungkan efek proporsional, integral, dan derivatif dari keluaran sistem y . Dengan menotasikan ulang pada state space pada persamaan (9) dan (10) dan mengasumsikan kontrol u adalah keluaran dari PID controller dengan input y

$$u = K_I \int_0^T y dt + K_P y + K_D \dot{y} \quad (13)$$

Di mana,

K_I, K_P, K_D merupakan parameter untuk gain integral, proporsional, dan derivatif.

2) Umpan Balik State

Kontrol PID pada persamaan (13) akan direpresentasikan sebagai hukum umpan balik state sebagai berikut.

$$y = C.x \quad (14)$$

$$\dot{y} = CA.x + CB.u \quad (15)$$

$$\dot{y} = CA^2.x + CAB.u + CB.\dot{u} \quad (16)$$

Dengan mensubstitusi persamaan (14), (15), dan (16) ke persamaan (13), akan diperoleh persamaan sebagai berikut.

$$(1 - K_D CB)\dot{u} - (K_D CA^2 + K_P CA + K_I C)x - (K_D CAB + K_P CB)u = 0 \quad (17)$$

Dengan menormalisasikan K^T menjadi $\hat{K}^T$ sebagai berikut.

$$\hat{K}^T = [\hat{K}_I \quad \hat{K}_P \quad \hat{K}_D] = (1 - K_D CB)^{-1} [K_I \quad K_P \quad K_D]^T \quad (18)$$

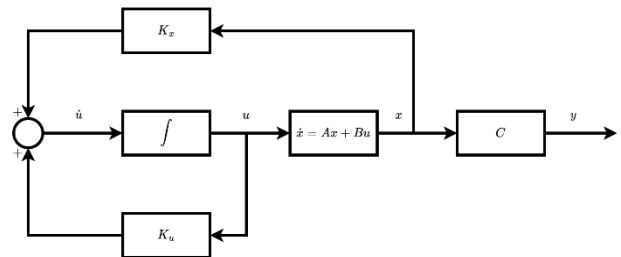
Persamaan (17) menjadi,

$$\dot{u} = \hat{K}^T [C^T \quad A^T C^T \quad (A^T)^2 C^T]^T x + \hat{K}^T [0 \quad B^T C^T \quad B^T A^T C^T]^T u \quad (19)$$

Di mana, didefinisikan sebagai berikut

- $K_x = \hat{K}^T [C^T \quad A^T C^T \quad (A^T)^2 C^T]^T$
- $K_u = \hat{K}^T [0 \quad B^T C^T \quad B^T A^T C^T]^T$

Agar lebih mudah memahami kontrol PID dapat dilihat pada diagram blok di Gambar 2. Dalam representasi umpan balik status, dapat dilihat bahwa kontroler PID memiliki fitur menarik. Ia memiliki umpan balik status di loop atas dan integrator murni di loop bawah. Dengan menggunakan prinsip model internal (*Internal Model Principle*, IMP), backstepping di loop bawah juga memastikan bahwa pengendali PID akan menghasilkan kesalahan pelacakan nol untuk sinyal referensi langkah.



Gambar 2. Diagram Blok Kontrol PID Representasi State Space

Persamaan (19) adalah hukum umpan balik keluaran yang menggunakan umpan balik status yang dibatasi. Secara khusus, hukum kontrol ini adalah hukum umpan balik status untuk sistem (9) dan (10) yang diperluas dengan integrator sebagai masukan.

Perhatikan bahwa sinyal kontrol dalam persamaan (13) dapat ditulis dengan integrator sebagai masukan.

$$\mathbf{u}_a = \mathbf{K}_a^T \mathbf{x}_a \quad (20)$$

Di mana,

$$\bullet \mathbf{u}_a = \dot{\mathbf{u}}$$

$$\bullet \mathbf{x}_a = \begin{bmatrix} \mathbf{x} \\ \mathbf{u} \end{bmatrix}$$

$$\mathbf{K}_a = \begin{bmatrix} \mathbf{C}^T & \mathbf{A}^T \mathbf{C}^T & (\mathbf{A}^T)^2 \mathbf{C}^T \\ \mathbf{0} & \mathbf{B}^T \mathbf{C}^T & \mathbf{B}^T \mathbf{A}^T \mathbf{C}^T \end{bmatrix} \hat{\mathbf{K}} = \mathbf{\Gamma} \hat{\mathbf{K}} \quad (21)$$

3) Augmentasi State Space

Persamaan augmentasi sistem oleh persamaan (9) dan (21) adalah sebagai berikut.

$$\dot{\mathbf{x}}_a = \mathbf{A}_a \mathbf{x}_a + \mathbf{B}_a \mathbf{u}_a \quad (22)$$

Di mana,

$$\bullet \mathbf{A}_a = \begin{bmatrix} \mathbf{A} & \mathbf{B} \\ \mathbf{0} & \mathbf{0} \end{bmatrix}$$

$$\bullet \mathbf{B}_a = \begin{bmatrix} \mathbf{0} \\ \mathbf{I} \end{bmatrix}$$

Kontrol PID dapat direpresentasikan sebagai pengontrol berbasis state space dengan memperluas sistem asli menggunakan integrator pada masukan. Pendekatan ini membuat pengendali PID lebih fleksibel dan mampu menangani perubahan keluaran dengan lebih baik. Sistem ini dirancang untuk memastikan bahwa kesalahan pelacakan, yaitu selisih antara keluaran (y) dan sinyal referensi (r), dapat diminimalkan hingga nol menggunakan Prinsip Model Internal (*Internal Model Principle*, IMP).

Untuk meningkatkan kemampuan pelacakan, keluaran sistem (y) diubah menjadi kesalahan keluaran ($e = y - r$), di mana (r) adalah sinyal referensi. Pendekatan ini membuat PID bisa menangani berbagai sinyal referensi sambil tetap mempertahankan stabilitas dan keakuratan. Sistem yang diperluas ini sangat cocok untuk aplikasi yang membutuhkan respons cepat dan akurasi tinggi dalam mencapai target keluaran.

Namun, untuk menentukan parameter PID (K_p, K_I, K_D), matriks $\mathbf{\Gamma}$ dalam perhitungan harus tidak singular (dapat dibalik). Jika tidak, metode optimasi diperlukan untuk menemukan nilai parameter yang optimal. Dengan metode ini, pengendali PID dapat dirancang secara optimal meskipun terdapat kendala matematis dalam perhitungan langsung. Pendekatan ini memastikan pengendali tetap stabil dan bekerja dengan performa terbaik.

B. LQR [3]

1) Fungsi Biaya LQR

Fungsi biaya J yang akan diminimalkan dalam LQR adalah kombinasi kesalahan state dan energi kontrol, yaitu.

$$J = \int_0^\infty (\mathbf{x}^T \mathbf{Q} \mathbf{x} + \mathbf{u}^T \mathbf{R} \mathbf{u}) dt \quad (23)$$

Di mana,

• $\mathbf{Q} \geq 0$ adalah matriks bobot untuk kesalahan state (diinginkan kecil).

• $\mathbf{R} > 0$ adalah matriks bobot untuk input kontrol (diinginkan kecil).

2) Kontrol LQR

Kontrol optimal $\mathbf{u}$ adalah fungsi linear dari state $\mathbf{x}$, yaitu.

$$\mathbf{u} = -\mathbf{K} \cdot \mathbf{x} \quad (24)$$

Di mana

• $\mathbf{K}$ adalah matriks gain yang dicari

3) Matriks Gain $\mathbf{K}$

Matriks gain $\mathbf{K}$ dihitung dengan menggunakan persamaan **Riccati**. Solusi dari persamaan Riccati memberikan matriks $\mathbf{P}$, yang digunakan untuk menentukan $\mathbf{K}$.

$$\mathbf{K} = \mathbf{R}^{-1} \cdot \mathbf{B}^T \cdot \mathbf{P} \quad (25)$$

Persamaan Riccati untuk matriks $\mathbf{P}$ adalah sebagai berikut.

$$\mathbf{A}^T \cdot \mathbf{P} + \mathbf{P} \cdot \mathbf{A} - \mathbf{P} \cdot \mathbf{B} \cdot \mathbf{R}^{-1} \cdot \mathbf{B}^T \cdot \mathbf{P} + \mathbf{Q} = 0 \quad (26)$$

Pada MATLAB, ada fungsi bawaan yang disebut *lqr* untuk menghitung matriks gain $\mathbf{K}$ dalam desain kontrol LQR. Fungsi ini secara otomatis menyelesaikan masalah LQR dengan menggunakan persamaan Riccati untuk menghasilkan matriks gain $\mathbf{K}$ yang optimal.

4) Stabilitas

Sistem tertutup yang diperoleh dari $\dot{\mathbf{x}} = (\mathbf{A} - \mathbf{B}\mathbf{K})\mathbf{x}$ akan stabil jika $\mathbf{A} - \mathbf{B}\mathbf{K}$ memiliki eigenvalue dengan bagian real negatif, yang memastikan bahwa sistem mencapai keseimbangan yang stabil.

IV. GENETIC ALGORITHM [3]

Genetic Algorithm (GA) digunakan untuk menemukan nilai optimal untuk matriks $\mathbf{Q}$ dan $\mathbf{R}$ pada kontrol LQR. Nilai-nilai dari GA adalah parameter yang mengontrol seberapa besar pengaruh state error $\mathbf{Q}$ dan input $\mathbf{R}$ terhadap biaya keseluruhan.

Tujuan dari optimasi ini adalah untuk meminimalkan fungsi biaya GA, yang didasarkan pada state error, overshoot, waktu settling, dan error steady-state.

A. Fungsi Biaya GA

1) State Error

State error mengukur perbedaan antara trajektor state dan nilai referensi. State error untuk setiap state i pada waktu t dapat dituliskan sebagai berikut.

$$e_i = x_i - r_i \quad (27)$$

Karena terdapat 4 state, state error $\mathbf{E}$ dapat dituliskan secara vektor sebagai berikut.

$$\mathbf{E} = \begin{bmatrix} e_1 \\ e_2 \\ e_3 \\ e_4 \end{bmatrix} = \begin{bmatrix} x_1 - r_1 \\ x_2 - r_2 \\ x_3 - r_3 \\ x_4 - r_4 \end{bmatrix} \quad (28)$$

Fungsi biaya utama yang dihitung dalam GA adalah integrasi kuadrat state error sepanjang waktu simulasi T .

$$\begin{aligned} Cost_{state} &= \int_0^T ||\mathbf{E}||^2 dt \\ &= \int_0^T (e_1^2 + e_2^2 + e_3^2 + e_4^2) dt \end{aligned} \quad (29)$$

Di mana

- T adalah waktu simulasi

2) Overshoot

Overshoot mengukur seberapa besar respons sistem melebihi nilai steady-state setelah gangguan. Overshoot untuk setiap state i dihitung sebagai persentase deviasi dari nilai puncak respons terhadap nilai steady state.

$$Overshoot_i = \frac{Peak_i - Steady_i}{Steady_i} \times 100 \quad (30)$$

Di mana,

- $Peak_i$ adalah nilai puncak dari respons state i
- $Steady_i$ adalah nilai steady-state dari state i

3) Settling Time

Settling time mengukur waktu yang dibutuhkan oleh sistem untuk tetap berada dalam kisaran tertentu (pada sistem ini 2% dari nilai steady-state) setelah gangguan. Settling time untuk state i dihitung sebagai waktu yang dibutuhkan untuk mencapai kondisi ini.

$$Settling Time_i = \min \{t: |x_i - Steady_i| \leq 0.02 \times |Steady_i|\} \quad (31)$$

Jika sistem tidak mencapai batas toleransi dalam rentang waktu simulasi, settling time dianggap tidak terdefinisi.

4) Steady-State Error

Steady-state error mengukur perbedaan antara nilai steady-state dan nilai referensi setelah sistem mencapai keadaan stabil. Steady-state error untuk state i adalah selisih antara nilai steady-state dan referensi.

$$Steady State Error_i = |Steady_i - ref_i| \quad (32)$$

5) Fungsi Biaya GA Total

Fungsi biaya total J adalah penjumlahan dari beberapa komponen biaya yang mencakup state error, overshoot, settling time, dan steady-state error. Secara matematis, fungsi biaya total dituliskan sebagai berikut.

$$\begin{aligned} J &= \int_0^T (e_1^2 + e_2^2 + e_3^2 + e_4^2) dt + \sum_{i=1}^4 Overshoot_i \\ &\quad + \sum_{i=1}^4 Settling Time_i \\ &\quad + 100 \sum_{i=1}^4 Steady State Error_i \end{aligned} \quad (33)$$

Nilai 100 yang dikalikan dengan error steady-state adalah untuk memberikan penalti yang lebih besar terhadap error steady state, karena hal ini sangat memengaruhi performa sistem dalam jangka panjang

Fungsi biaya tersebut dioptimalkan oleh GA untuk mencari matriks $\mathbf{Q}$ dan $\mathbf{R}$ yang menghasilkan $\mathbf{K}$ optimal, yaitu kontrol umpan balik dari sistem.

B. Batas Ruang Pencarian GA

Batas ruang pencarian untuk parameter $\Psi = [q_1 \ q_2 \ q_3 \ q_4 \ q_5 \ q_6 \ r_1 \ r_2]$ terdiri dari batas bawah (**lb**) yang mengatur nilai minimum dan batas atas (**ub**) yang mengatur nilai maksimum untuk setiap elemen dari $\mathbf{Q}$ dan $\mathbf{R}$.

Berikut adalah nilai *lower bounds lb* dan *upper bounds ub*

$$\begin{aligned} \mathbf{lb} \\ = [0.1 \ 0.1 \ 0.1 \ 0.1 \ 0.1 \ 0.1 \ 0.01 \ 0.01] \end{aligned} \quad (34)$$

$$\begin{aligned} \mathbf{ub} \\ = [0.1 \ 0.1 \ 0.1 \ 0.1 \ 0.1 \ 0.1 \ 0.01 \ 0.01] \end{aligned} \quad (35)$$

C. Inisialisasi dan Pengaturan GA

Genetic algorithm diinisialisasi dengan parameter sebagai berikut.

- **MaxGenerations:** 1000 iterasi, jumlah maksimal generasi dalam pencarian solusi.
- **PopulationSize:** 20 individu dalam populasi pada setiap generasi.

D. Algoritma GA

Sebelum algoritma bekerja, populasi awal dibuat secara acak dalam jangkauan batas yang telah ditentukan. Rentang batas ini terletak antara batas bawah **lb** dan batas atas **ub**. Setelah itu, pemilihan individu dilakukan dengan menggunakan teknik seperti *tournament selection* atau *roulette wheel selection*. Penyilangan, atau *crossover*, adalah proses menggabungkan informasi genetik dari dua *parent* untuk membentuk individu baru. Penggabungan dilakukan dengan mengambil bagian dari *parent* dan mencampurnya dengan rumus.

$$\Psi_{child} = \alpha \Psi_{parent1} + (1 - \alpha) \Psi_{parent2} \quad (36)$$

Di mana,

- $\alpha \in [0,1]$ adalah faktor pencampuran yang mengatur seberapa banyak kontribusi dari masing-masing individu (*parent*) dalam membentuk individu baru (*child*).

Selain penyilangan, proses mutasi juga dilakukan agar tidak terjadi konvergensi optimasi secara prematur.

$$\Psi_{mutan} = \Psi + \psi \quad (37)$$

Di mana,

- ψ adalah gangguan kecil yang dihasilkan secara acak

Untuk mengevaluasi seleksi dan penghasilan generasi individu baru ini, fungsi biaya $J(\Psi)$ akan digunakan. Jika populasi yang dihasilkan tidak memenuhi syarat atau kinerjanya buruk, populasi tersebut dimusnahkan dan digantikan dengan populasi baru. Sampai kriteria konvergensi terpenuhi dan tidak ada peningkatan nilai yang signifikan pada fungsi biaya, prosedur ini diulang.

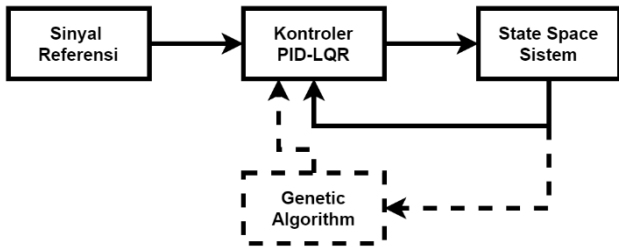
V. HASIL DAN PEMBAHASAN

Pada simulasi ini, dua jenis skenario yang berbeda diuji untuk mengoptimalkan performa sistem suspensi kendaraan menggunakan kontrol LQR.

- **Skenario dasar**, Kontrol LQR dengan $Q = \text{diag}(1, 1, 1, 1, 1, 1, 1)$ dan $R = \text{diag}(1, 1)$, yang digunakan sebagai kontrol dasar tanpa optimasi.
- **Skenario GA**, Kontrol LQR dengan Q dan R yang sudah dioptimalkan menggunakan algoritma **Genetic Algorithm (GA)**

Simulasi dilakukan dengan menggunakan MATLAB untuk mendefinisikan parameter yang digunakan dalam model simulink, serta untuk mengimplementasikan algoritma GA. Sedangkan, Simulink digunakan untuk mensimulasikan dinamika sistem berdasarkan kontrol yang diterapkan selama durasi 30 detik.

Secara umum, program simulasi dengan blok diagram sebagai berikut.



Gambar 3 Blok Diagram Program Simulasi

A. Define Parameter di Matlab

Pada simulasi ini akan digunakan state space suspensi kendaraan yang sudah di augmentasi sesuai dengan persamaan (22) dan (21) sebagai berikut.

$$A_a = \begin{bmatrix} A & B \\ \text{zeros}(2,4) & \text{zeros}(2,2) \end{bmatrix} \quad (38)$$

$$B_a = \begin{bmatrix} \text{zeros}(4,2) \\ \text{eye}(2,2) \end{bmatrix} \quad (39)$$

$$\Gamma = \begin{bmatrix} C & \text{zeros}(2,2) \\ CA & CB \\ CA^2 & CAB \end{bmatrix} \quad (40)$$

Matriks A, B, C dapat dilihat di persamaan (41), (42), dan (43) dengan parameter k_k, k_r, m_k, b_r, m_c dapat dilihat pada Tabel 1.

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ -\frac{k_k - k_r}{m_k} & -\frac{b_r}{m_k} & \frac{k_r}{m_k} & \frac{b_r}{m_k} \\ 0 & 0 & 0 & 1 \\ k_r & m_c & -\frac{k_r}{m_c} & -\frac{b_r}{m_c} \end{bmatrix} \quad (41)$$

$$B = \begin{bmatrix} 0 & 0 \\ -\frac{1}{m_k} & \frac{k_k}{m_k} \\ 0 & 0 \\ \frac{1}{m_c} & 0 \end{bmatrix} \quad (42)$$

$$C = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (43)$$

Parameter K_P, K_I , dan K_D dapat dicari dengan mendekomposisi K_a pada persamaan (21). Persamaan (21) ini diperoleh dari proses penurunan K pada kontrol LQR (sebut saja K_{LQR}) untuk state space yang sudah diaugmentasi.

Berikut adalah penurunannya.

$$\hat{K} = K_{LQR}(\Gamma)^{-1} \quad (44)$$

$$\hat{K}_D = \hat{K}(1,5:6)$$

$$K_D = \frac{\hat{K}_D}{1 + \hat{K}_D C B} \quad (45)$$

$$\hat{K}_P = \hat{K}(1,3:4)$$

$$K_P = \hat{K}_P(1 - K_D C B) \quad (46)$$

$$\hat{K}_I = \hat{K}(1,1:2)$$

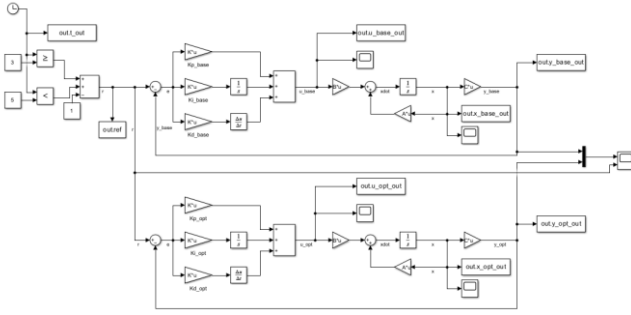
$$K_I = \hat{K}_I(1 - K_D C B) \quad (47)$$

Setelah seluruh parameter sudah dijalankan dan disimpan pada workspace matlab, simulasikan sistem dengan menjalankan simulink.

B. Simulasi Sistem di Simulink

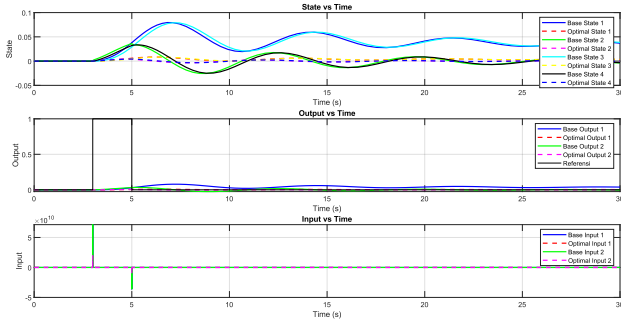
Sinyal referensi yang diberikan pada sistem suspensi kendaraan berupa sinyal step yang aktif pada rentang waktu 3 hingga 5 detik, sementara di luar periode tersebut bernilai nol. Dalam konteks ini, sinyal step tersebut menggambarkan kehadiran *speed bump* yang dilalui oleh kendaraan, lalu dapat dilihat bagaimana sistem suspensi merespons *speed bump* tersebut.

Pada saat sinyal step aktif (3 hingga 5 detik), aktuator pada sistem suspensi akan berusaha untuk menyesuaikan posisi kendaraan (y_c) dan kecepatan vertikal kendaraan ($\dot{y}_c$) guna mengurangi dampak dari gangguan tersebut. Tujuannya adalah untuk menjaga kenyamanan penumpang dan kestabilan kendaraan dengan mengurangi osilasi atau pergerakan yang berlebihan akibat *speed bump*. Sistem kontrol akan bekerja untuk meminimalkan perbedaan antara posisi dan kecepatan kendaraan terhadap referensi yang diberikan, serta untuk meredam efek yang ditimbulkan oleh gangguan tersebut.



Gambar 4. Blok Diagram Sistem Suspensi Kendaraan

Setelah dijalankan sistem blok diagram pada Gambar 4 selama 30 detik, diperoleh respon sistem y_c dan $\dot{y}_c$ sebagai berikut.



Gambar 5. Plot Respons y_c dan $\dot{y}_c$

Bedasarkan plot yang diperoleh pada Gambar 5 **Error! Reference source not found.** menunjukkan respon sistem berupa plot yang menampilkan perbandingan antara state sistem untuk kedua kontroler pada 4 state yang berbeda, plot yang menampilkan perbandingan antara output sistem untuk kedua kontroler dan referensi, dan plot yang menampilkan perbandingan antara input kontrol untuk kedua kontroler.

Pada grafik state, menunjukkan respons masing-masing state terhadap *speed bump*. Pada kontroler optimal (dengan optimasi genetika), amplitudo osilasi pada state lebih kecil dibandingkan kontroler dasar, menunjukkan bahwa optimasi genetika membantu meredam getaran lebih efektif. Settling time untuk kontroler optimal juga lebih cepat dibandingkan kontroler dasar, yang menandakan waktu stabilitas sistem lebih singkat.

Pada grafik output, respons menggambarkan perpindahan vertikal badan kendaraan (y_c) dan kecepatan vertikal badan kendaraan ($\dot{y}_c$). Output dari kontroler optimal lebih dekat dengan kondisi semula, dengan overshoot dan error transien yang lebih kecil. Respons kontroler dasar memiliki fluktuasi yang lebih signifikan sebelum mencapai kestabilan.

Pada grafik input respons menggambarkan sinyal kendali yang dihasilkan untuk mencapai kestabilan. Pada kontroler optimal, sinyal kendali lebih halus dan stabil setelah gangguan, sedangkan kontroler dasar menunjukkan puncak input yang lebih besar di awal respons.. Efisiensi input pada kontroler optimal juga lebih baik, yang berarti konsumsi energi atau usaha kontrol lebih rendah.

Pada simulasi, kontrol base menunjukkan steady-state error yang besar dan tidak kembali ke nol, sementara kontrol yang dioptimalkan dengan GA mampu mengurangi error tersebut dan kembali ke nol setelah gangguan. Selain itu,

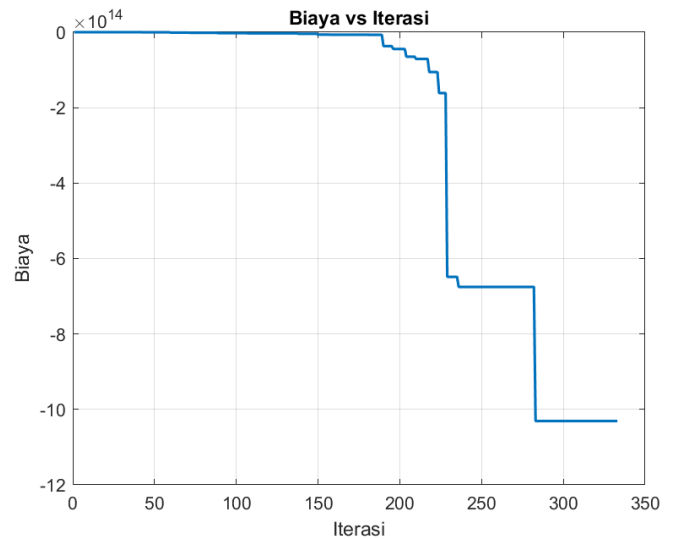
waktu settling untuk kontrol GA sangat cepat dan hampir tanpa overshoot, menunjukkan respons yang efisien dan stabil terhadap gangguan. Sebaliknya, kontrol dasar membutuhkan waktu lebih lama untuk stabil dan menunjukkan overshoot yang lebih besar. Hasil ini mengindikasikan bahwa optimasi menggunakan GA memberikan kinerja yang jauh lebih baik dibandingkan kontrol dasar.

Untuk kontroler PID-LQR dasar, nilai Q dan R diatur secara tetap, yaitu $Q = \text{diag}([1, 1, 1, 1, 1, 1])$ dan $R = \text{diag}([1, 1])$, dan kontroler dihitung dengan menggunakan fungsi *lqr* yang menghasilkan matriks K_{base} . Matriks K_{base} ini digunakan untuk mengatur input kontrol untuk meminimalkan error status sistem. Berikut adalah nilai K_{base} yang digunakan.

$$K_{base} = \begin{bmatrix} -0.0113 & -8.3620 \times 10^{-5} & 0.0106 & 0.0025 & 1.0000 & -5.8968 \times 10^{-5} \\ -0.7543 & 0.1840 & 0.5914 & 1.7251 & -5.8968 \times 10^{-5} & 1.8950 \end{bmatrix} \quad (48)$$

Untuk kontroler PID-LQR dengan optimasi genetika, nilai Q dan R dicari secara optimal dengan menggunakan algoritma genetika. Dalam hal ini, algoritma genetika akan mengoptimalkan parameter Q dan R untuk mendapatkan kontroler yang dapat meminimalkan biaya atau error dalam sistem. Proses optimasi ini dilakukan dengan mengubah parameter Q dan R untuk mencapai nilai yang memberikan performa terbaik untuk kontroler.

Berikut biaya yang dihitung selama optimasi genetika untuk menemukan nilai optimal Q dan R .



Gambar 6. Biaya untuk Mencari Parameter Optimal

Biaya pada Gambar 6 menunjukkan bagaimana performa kontroler berkembang selama iterasi optimasi. Setelah nilai optimal Q_{opt} dan R_{opt} ditemukan, kontroler LQR dihitung kembali dengan menggunakan fungsi *lqr* untuk mendapatkan K_{opt} , yang digunakan dalam simulasi untuk membandingkan performa antara kontroler dasar dan kontroler optimal. Berikut adalah nilai parameter yang sudah dioptimalkan.

$$Q_{opt} = \begin{bmatrix} 87.4999 & 0 & 0 & 0 & 0 & 0 \\ 0 & 47.9164 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0.5189 & 0 & 0 & 0 \\ 0 & 0 & 0 & 63.4103 & 0 & 0 \\ 0 & 0 & 0 & 0 & 99.5451 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0.9337 \end{bmatrix}$$

$$R_{opt} = \begin{bmatrix} 5.3421 & 0 \\ 0 & 9.3203 \end{bmatrix}$$

$$K_{opt} = \begin{bmatrix} -0.0026 & -3.3166 \times 10^{-4} & 3.0339 \times 10^{-4} & -0.0015 & 4.3167 & -4.9241 \times 10^{-4} \\ 0.3735 & 0.4810 & 0.0944 & 3.8101 & -2.8223 \times 10^{-4} & 2.6215 \end{bmatrix}$$

VI. KESIMPULAN

Penerapan Genetic Algorithm (GA) sebagai metode tuning kontrol pada PID-LQR telah berhasil meningkatkan performa sistem suspensi kendaraan. Dengan menggunakan GA, nilai Q_{opt} dan R_{opt} yang diperoleh adalah $Q_{opt} = \text{diag}(87.4999, 47.9164, 0.5189, 63.4103, 99.5451, 0.9337)$ dan $R_{opt} = \text{diag}(5.3421, 9.3203)$.

Hasil simulasi menunjukkan bahwa kontrol yang dioptimalkan dengan GA mengurangi steady-state error secara signifikan, mengurangi overshoot, dan mempercepat settling time dibandingkan dengan kontrol base yang menggunakan nilai $Q = \text{diag}(1, 1, 1, 1, 1, 1)$ dan $R = \text{diag}(1, 1)$.

GA berhasil memberikan penalti pada state error, overshoot, settling time, dan steady-state error, sehingga menghasilkan sistem yang lebih stabil dan responsif terhadap gangguan. Dengan demikian, optimasi menggunakan GA terbukti efektif dalam meningkatkan kinerja kontrol PID-LQR pada sistem suspensi kendaraan.

VII. DAFTAR PUSTAKA

- [1] K. Á. Kis, P. Korondi, G. Korsoveczki, I. Kocsis, K. Sarvajcz dan I. Balajti, "Quarter car suspension state space model and full state feedback control for Real-Time Processing," 2023.
- [2] E. Joelianto, "Linear Quadratic Control: A State Space Approach," *Automatic Control Engineering: Teknik Kontrol Otomatis*, 2017.
- [3] I. A. Herdian, I. Bintang dan A. Nurfarid, "Perancangan Simulasi dan Analisis Kinerja Sistem Kontrol Optimal pada Active Quarter Car Suspension Orde-4," 2024.

VIII. LAMPIRAN

A. Source Code Matlab

```
clear
clc
close all

%%
% Parameter Front Side (BMW)
kk = 340;      % kN/m
kr = 30;       % kN/m
br = 1450;     % Ns/m
mc = 408;      % kg
mus = 48.3;    % kg

%%
% State-space
% Matriks state-space
A = [0 1 0 0;
     (-kk-kr)/mus -br/mus kr/mus br/mus;
     0 0 0 1;
     kr/mc br/mc -kr/mc -br/mc];

B1 = [0;
      -1/mus;
      0;
      1/mc];

B2 = [0;
      kk/mus;
      0;
      0];

B = [B1 B2];

C = [0 0 1 0;
     0 0 0 1];

D = [0 0
```

```

0 0];

%%
% Augmentasi Backstepping Integral
A_aug = [A B;
         zeros(2, 4) zeros(2,2)];
B_aug = [zeros(4, 2);
         eye(2,2)];

gamma = [C zeros(2, 2);
         C*A C*B;
         C*A*A C*A*B];

%%
% Genetic Algorithm

% Batas untuk variabel optimasi
lb = [0.1 * ones(1, 6), 0.01 * ones(1, 2)]; % Batas bawah yang disesuaikan untuk Q dan R
ub = [100 * ones(1, 6), 10 * ones(1, 2)]; % Batas atas yang disesuaikan untuk Q dan R

% Inisialisasi variabel untuk menyimpan riwayat biaya
global cost_history;
cost_history = [];

% Jalankan algoritma genetika untuk mencari nilai Q dan R optimal
options = optimoptions('ga', 'MaxGenerations', 1000, 'PopulationSize', 20, ...
                      'Display', 'iter', 'OutputFcn', @ga_output_function);
optimal_params = ga(@(params) objective(params, A_aug, B_aug), 8, [], [], [], [], lb, ub, [],
options);

%%
% Desain Kontroler LQR Biasa
Q = diag([1, 1, 1, 1, 1, 1]);
R = diag([1, 1]);
[K_base, ~, ~] = lqr(A_aug, B_aug, Q, R);

K_hat_base = K_base*pinv(gamma);

% Parameter PID
K3_hat_base = K_hat_base(1, 5:6);
Kd_base = K3_hat_base/(1+K3_hat_base*C*B);
K2_hat_base = K_hat_base(1, 3:4);
Kp_base = K2_hat_base*(1-Kd_base*C*B);
K1_hat_base = K_hat_base(1, 1:2);
Ki_base = K1_hat_base*(1-Kd_base*C*B);

%%
% Desain Kontroler LQR Genertic Algorithm
Q_opt = diag(optimal_params(1:6));
R_opt = diag(optimal_params(7:8));
[K_opt, ~, ~] = lqr(A_aug, B_aug, Q_opt, R_opt);

K_hat_opt = K_opt*pinv(gamma);

% Parameter PID
K3_hat_opt = K_hat_opt(1, 5:6);
Kd_opt = K3_hat_opt/(1+K3_hat_opt*C*B);
K2_hat_opt = K_hat_opt(1, 3:4);
Kp_opt = K2_hat_opt*(1-Kd_opt*C*B);
K1_hat_opt = K_hat_opt(1, 1:2);
Ki_opt = K1_hat_opt*(1-Kd_opt*C*B);

%%
% Simulasi Menggunakan Simulink
stime = 30;
simOut = sim("CarilQR_PID.slx", stime);

```

```

% Ekstrak Data Waktu
t_out = simOut.t_out;

% Ekstrak Data Referensi
ref = simOut.ref;

% Ekstrak Data State
x_base_out = simOut.x_base_out;
x_opt_out = simOut.x_opt_out;

% Ekstrak Data Output
y_base_out = simOut.y_base_out;
y_opt_out = simOut.y_opt_out;

% Ekstrak Data Input
u_base_out = simOut.u_base_out;
u_opt_out = simOut.u_opt_out;

%%
% Plot Kontrol Base dan Opt

% Plot State (4 states)
figure;
subplot(3, 1, 1);
plot(t_out, x_base_out(:,1), 'b', 'LineWidth', 1.5); hold on;
plot(t_out, x_opt_out(:,1), 'r--', 'LineWidth', 1.5); % State 1
plot(t_out, x_base_out(:,2), 'g', 'LineWidth', 1.5);
plot(t_out, x_opt_out(:,2), 'm--', 'LineWidth', 1.5); % State 2
plot(t_out, x_base_out(:,3), 'c', 'LineWidth', 1.5);
plot(t_out, x_opt_out(:,3), 'y--', 'LineWidth', 1.5); % State 3
plot(t_out, x_base_out(:,4), 'k', 'LineWidth', 1.5);
plot(t_out, x_opt_out(:,4), 'b--', 'LineWidth', 1.5); % State 4
title('State vs Time');
xlabel('Time (s)');
ylabel('State');
legend('Base State 1', 'Optimal State 1', 'Base State 2', 'Optimal State 2', ...
'Base State 3', 'Optimal State 3', 'Base State 4', 'Optimal State 4');
grid on;

% Plot Output (2 outputs)
subplot(3, 1, 2);
plot(t_out, y_base_out(:,1), 'b', 'LineWidth', 1.5); hold on;
plot(t_out, y_opt_out(:,1), 'r--', 'LineWidth', 1.5); % Output 1
plot(t_out, y_base_out(:,2), 'g', 'LineWidth', 1.5);
plot(t_out, y_opt_out(:,2), 'm--', 'LineWidth', 1.5); % Output 2
plot(t_out, ref(:,1), 'black', 'LineWidth', 0.2); % Referensi
title('Output vs Time');
xlabel('Time (s)');
ylabel('Output');
legend('Base Output 1', 'Optimal Output 1', 'Base Output 2', 'Optimal Output 2', 'Referensi');
grid on;

% Plot Input (2 inputs)
subplot(3, 1, 3);
plot(t_out, u_base_out(:,1), 'b', 'LineWidth', 1.5); hold on;
plot(t_out, u_opt_out(:,1), 'r--', 'LineWidth', 1.5); % Input 1
plot(t_out, u_base_out(:,2), 'g', 'LineWidth', 1.5);
plot(t_out, u_opt_out(:,2), 'm--', 'LineWidth', 1.5); % Input 2
title('Input vs Time');
xlabel('Time (s)');
ylabel('Input');
legend('Base Input 1', 'Optimal Input 1', 'Base Input 2', 'Optimal Input 2');
grid on;

% Plot biaya vs iterasi
figure;
plot(cost_history, 'LineWidth', 1.5);

```

```

title('Biaya vs Iterasi');
xlabel('Iterasi');
ylabel('Biaya');
grid on;

%%
% Fungsi OutputFcn untuk merekam nilai biaya di setiap generasi
function [state, options, optchanged] = ga_output_function(options, state, flag)
    global cost_history;
    if strcmp(flag, 'iter')
        % Simpan biaya terbaik (fitness) pada generasi saat ini
        cost_history = [cost_history; min(state.Score)];
    end
    optchanged = false;
end

% Fungsi objektif untuk optimasi
function cost = objective(params, A, B)
    % Bentuk Q dan R dari parameter optimasi
    Q = diag(params(1:6)); % Disesuaikan untuk 4 status
    R = diag(params(7:8)); % Disesuaikan untuk 2 input

    % Hitung LQR
    K = lqr(A, B, Q, R);

    % Matriks sistem tertutup
    Acl = A - B * K;

    % Set kondisi awal
    x0 = [1; 1; 1; 1; 1; 1]; % Status awal

    % Simulasikan respons sistem tertutup
    [t, x] = ode45(@(t, x) Acl * x, [0, 10], x0);

    % Sinyal referensi adalah nol sepanjang simulasi
    r = zeros(size(x));

    % Hitung error status sepanjang periode waktu
    state_error = x - r; % Error tracking untuk semua status (pada dasarnya hanya x)

    % Hitung overshoot, waktu settling, dan error steady-state untuk setiap status
    overshoot = zeros(6, 1);
    settling_time = zeros(6, 1);
    steady_state_error = zeros(6, 1);

    for i = 1:6
        % Ambil trajektori status untuk status ini
        state_trajectory = x(:, i);

        % Hitung overshoot sebagai persentase deviasi puncak dari nilai steady-state
        steady_value = state_trajectory(end); % Nilai steady-state terakhir
        peak_value = max(state_trajectory); % Nilai puncak dari respons
        overshoot(i) = (peak_value - steady_value) / steady_value * 100;

        % Hitung waktu settling (waktu untuk tetap dalam 2% dari nilai steady-state)
        tolerance = 0.02 * abs(steady_value);
        idx_settling = find(abs(state_trajectory - steady_value) <= tolerance, 1, 'last');
        if ~isempty(idx_settling)
            settling_time(i) = t(idx_settling);
        else
            settling_time(i) = NaN; % Jika waktu settling tidak tercapai dalam rentang waktu
        end

        % Hitung error steady-state (nilai error terakhir)
        steady_state_error(i) = abs(state_error(end, i));
    end
end

```

```

% Integrasikan error kuadrat sepanjang waktu untuk perhitungan biaya
state_error_squared = sum(state_error.^2, 2); % Error kuadrat untuk setiap status
cost = trapz(t, state_error_squared); % Gunakan integrasi trapezoidal untuk luas di
bawah kurva

% Tambahkan overshoot, waktu settling, dan error steady-state ke biaya
% (Penalti untuk overshoot tinggi, waktu settling lama, dan error steady-state tinggi)
cost = cost + sum(overshoot) + sum(settling_time) + 100*sum(steady_state_error);
end

```

B. Diagram Block Simulink

